

Doubly Linked List – Search For Element And Time Complexity.

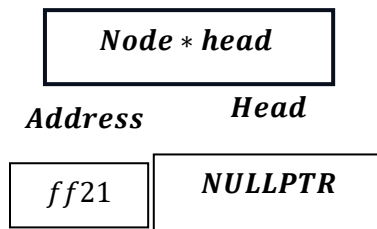
Program :

```
void searchForElement(int data)
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    int pos = 1;
    while (temp != nullptr)
    {
        if (temp->data == data)
        {
            cout << "Element " << data << " found at
position " << pos << "." << endl;
            return;
        }
        temp = temp->next;
        pos++;
    }
    cout << "Element " << data << " not found in the list."
<< endl;
}
```

Program Analysis

A. When LIST is Empty

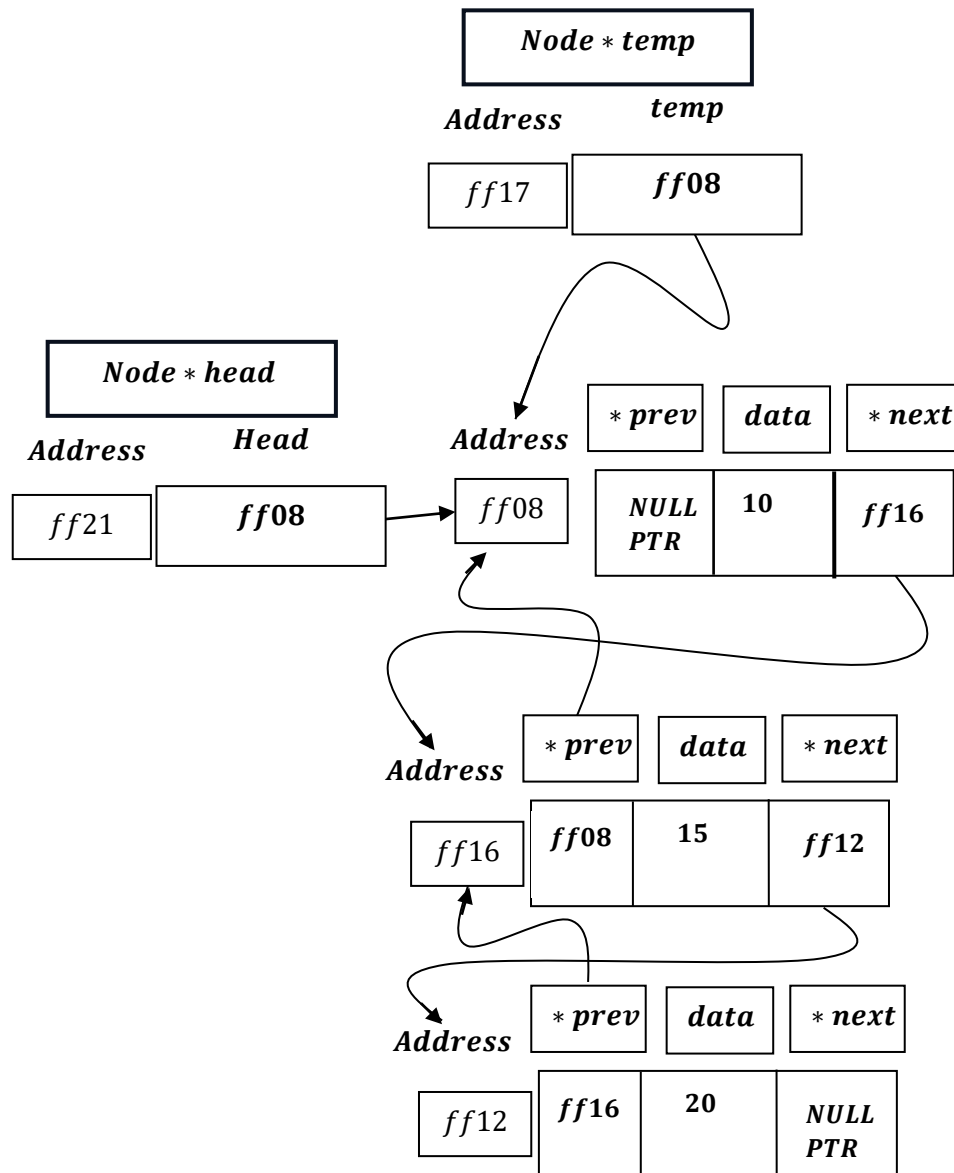
```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return;
}
```



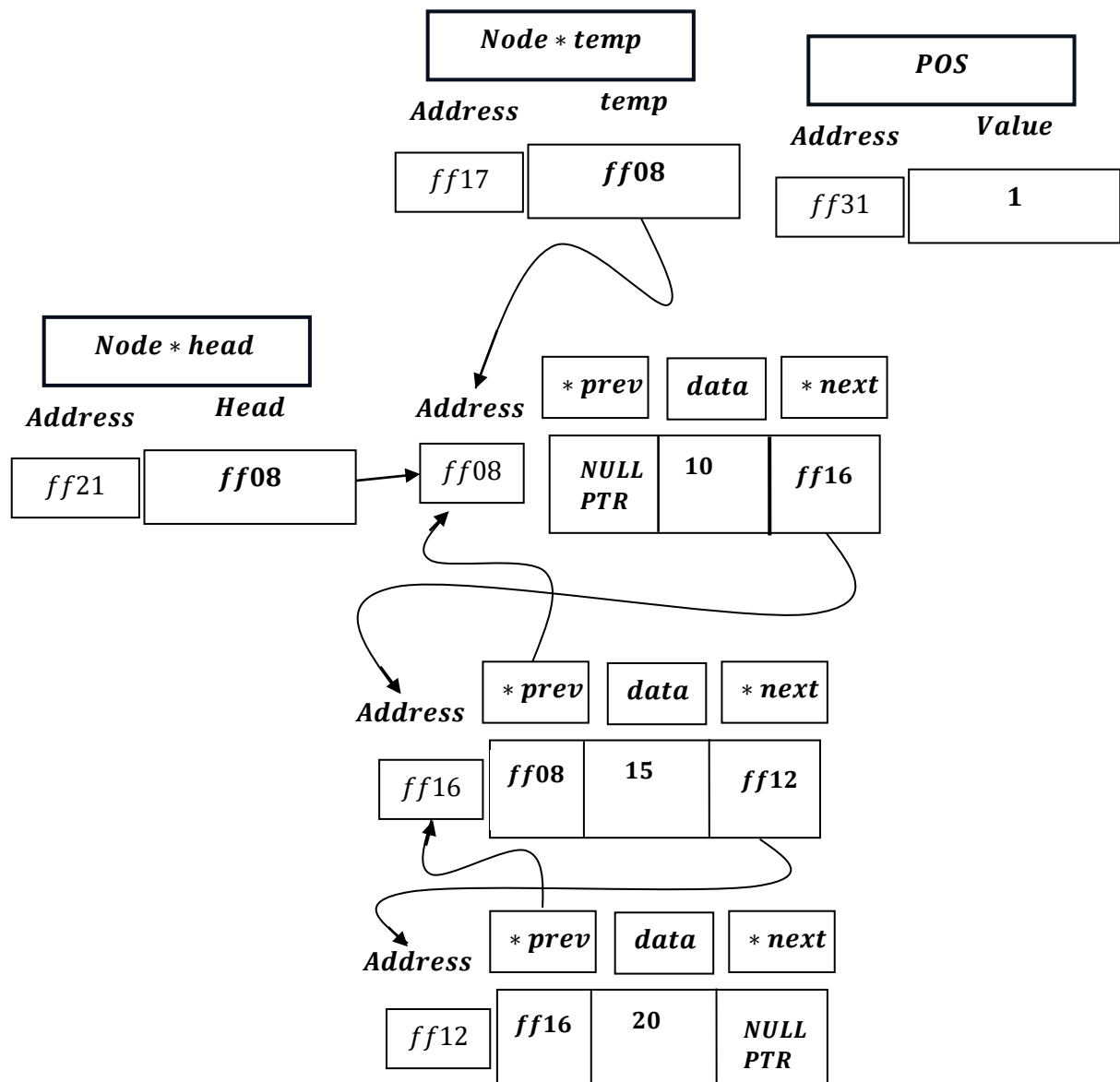
Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

```
Node *temp = head;
```



```
int pos = 1;
```



```

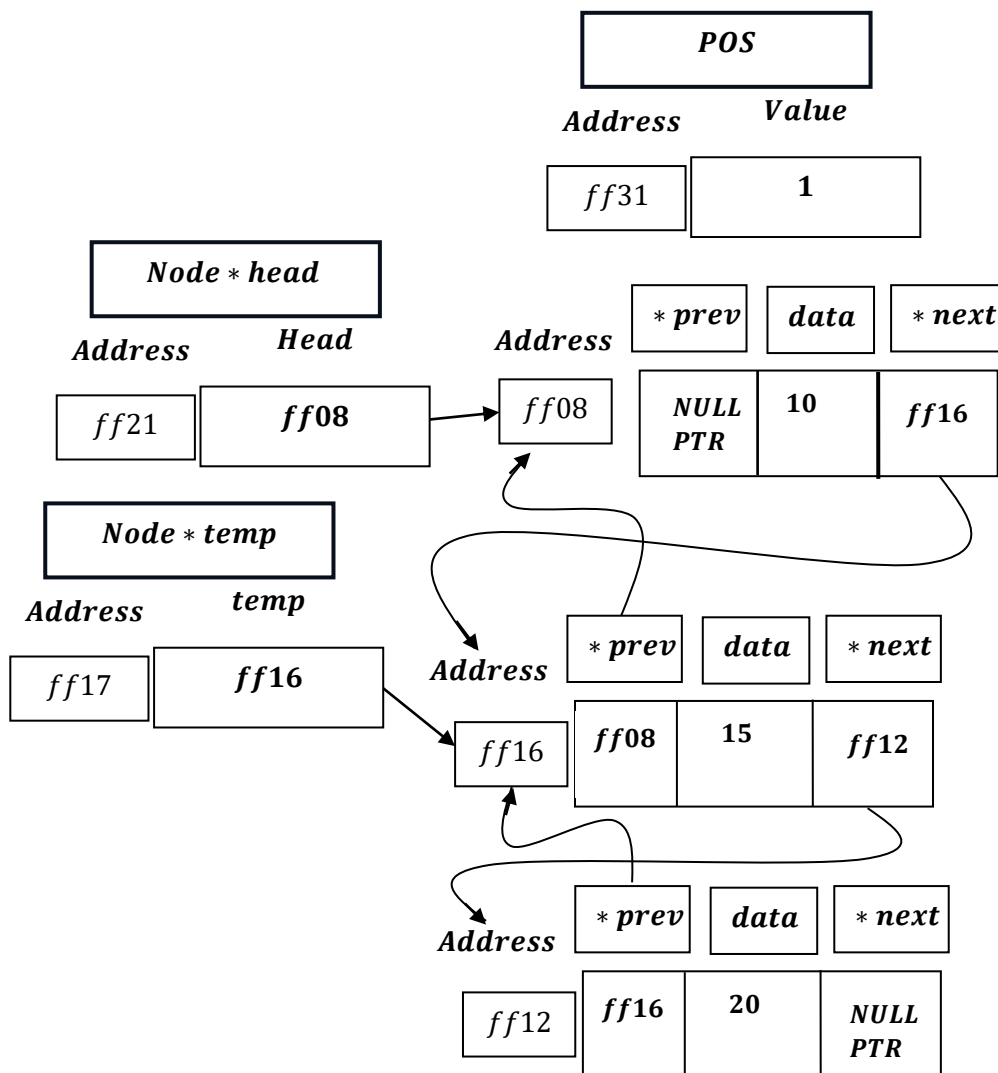
while (temp != nullptr)
{
    if (temp->data == data)
    {
        cout << "Element " << data << " found at
position " << pos << "." << endl;
        return;
    }
    temp = temp->next;
    pos++;
}

```

Let Data to find is : 20 ,Temp $\neq$ NULLPTR is true , hence:

$\rightarrow temp \rightarrow data = 10 \neq data = 20$.

$\rightarrow$ hence, as it is false , temp = temp $\rightarrow$ next = ff16 .

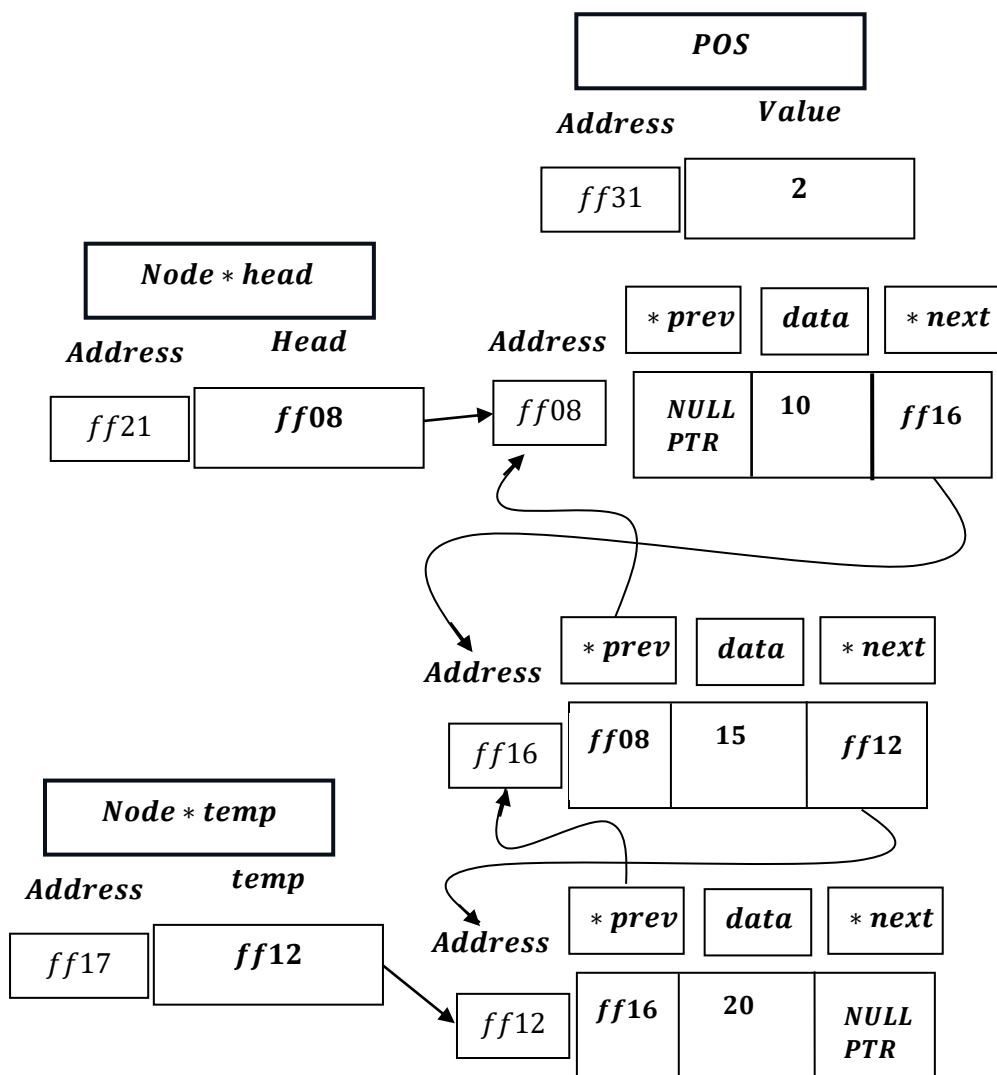


$$POS = POS + 1 = 1 + 1 = 2$$

Temp $\neq$ NULLPTR is true , hence:

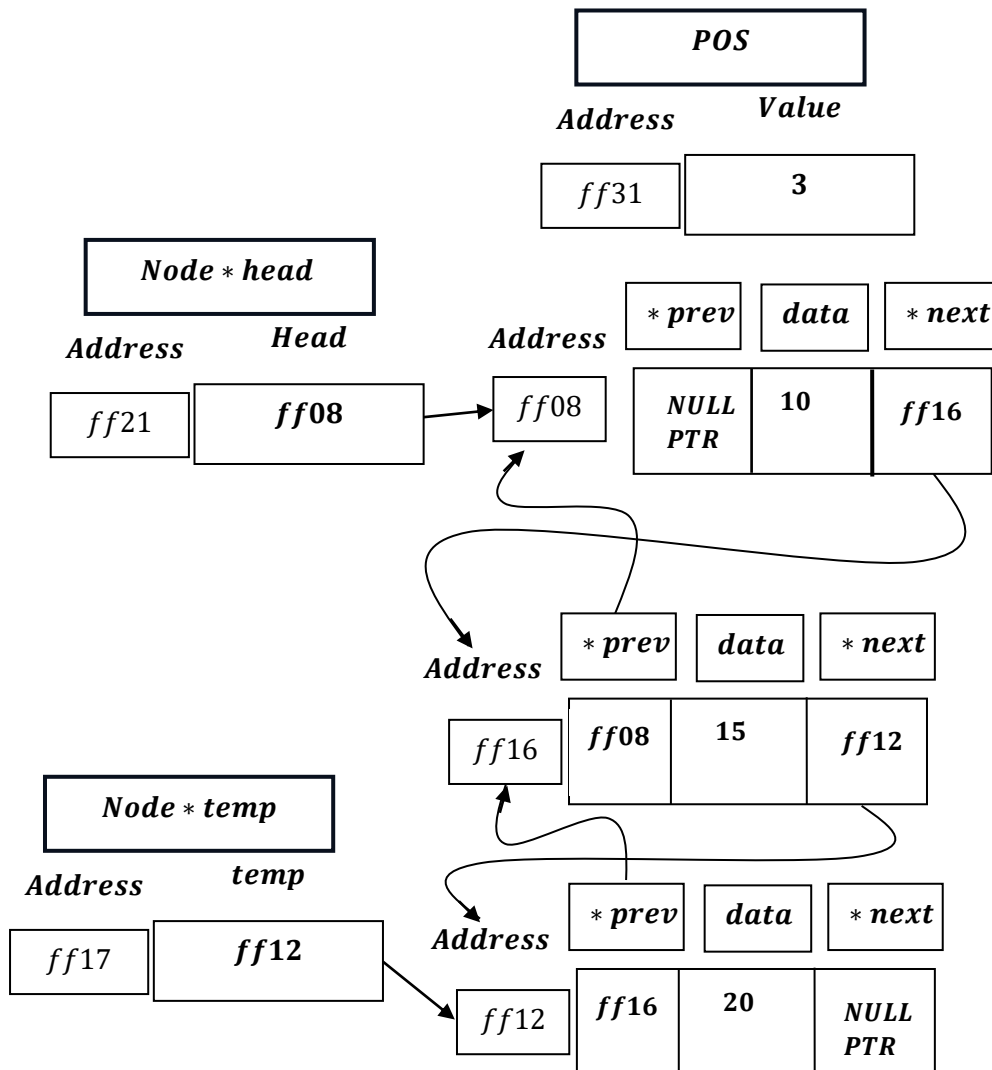
$\rightarrow temp \rightarrow data = 15 \neq data = 20$.

$\rightarrow temp = temp \rightarrow next = ff12$



$$POS = POS + 1 = 2 + 1 = 3$$

Temp $\neq$ NULLPTR is true , hence:



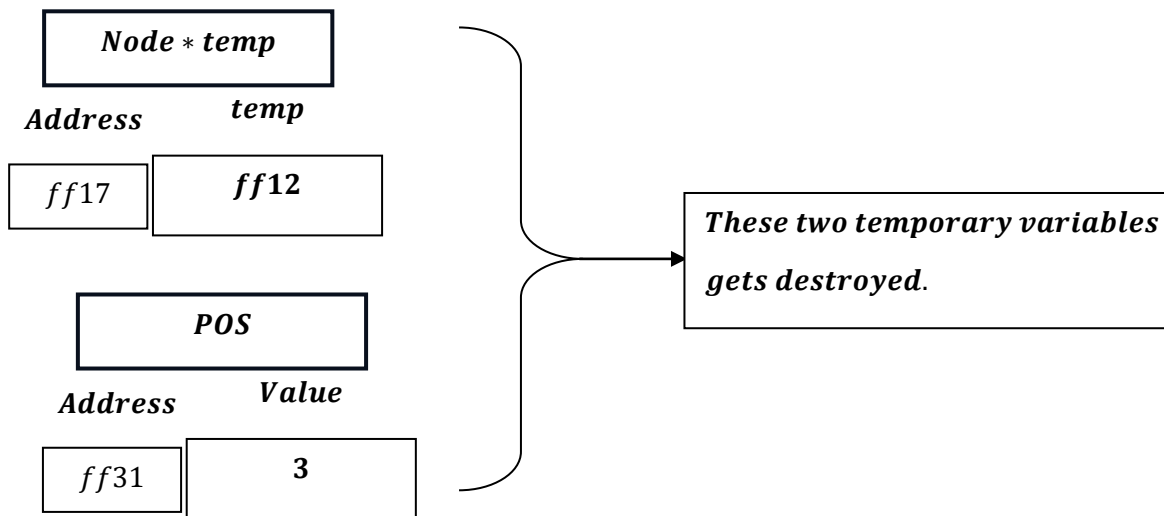
```
if (temp->data == data)
{
    cout << "Element " << data << " found at
position " << pos << "." << endl;
    return;
}
```

if temp → data = 20 = data = 20 is true:

→ It prints Element 20 found at position 3 .

→ Return statement makes exit from function.

As it exit from function Temp pointer along with pos variable gets destroyed:



Temp = NULLPTR ?

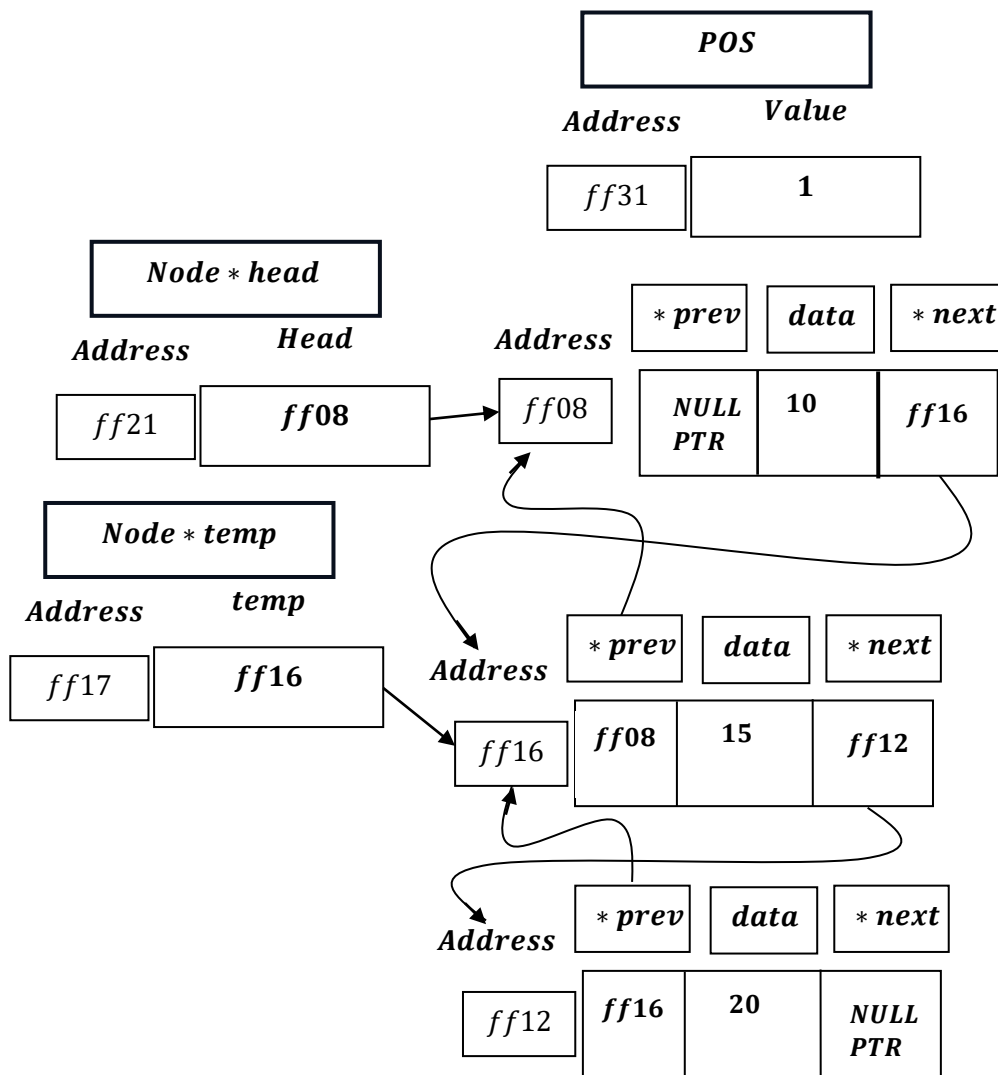
Q) When Temp = NULLPTR ? Ans: –When Data doesnot match with Data in nodes. Suppose data is 25 .

```
while (temp != nullptr)
{
    if (temp->data == data)
    {
        cout << "Element " << data << " found at
position " << pos << "." << endl;
        return;
    }
    temp = temp->next;
    pos++;
}
```

Let Data to find is : 25 ,Temp ≠ NULLPTR is true , hence:

→ temp → data = 10 ≠ data = 20.

→ hence, as it is false , temp = temp → next = ff16 .

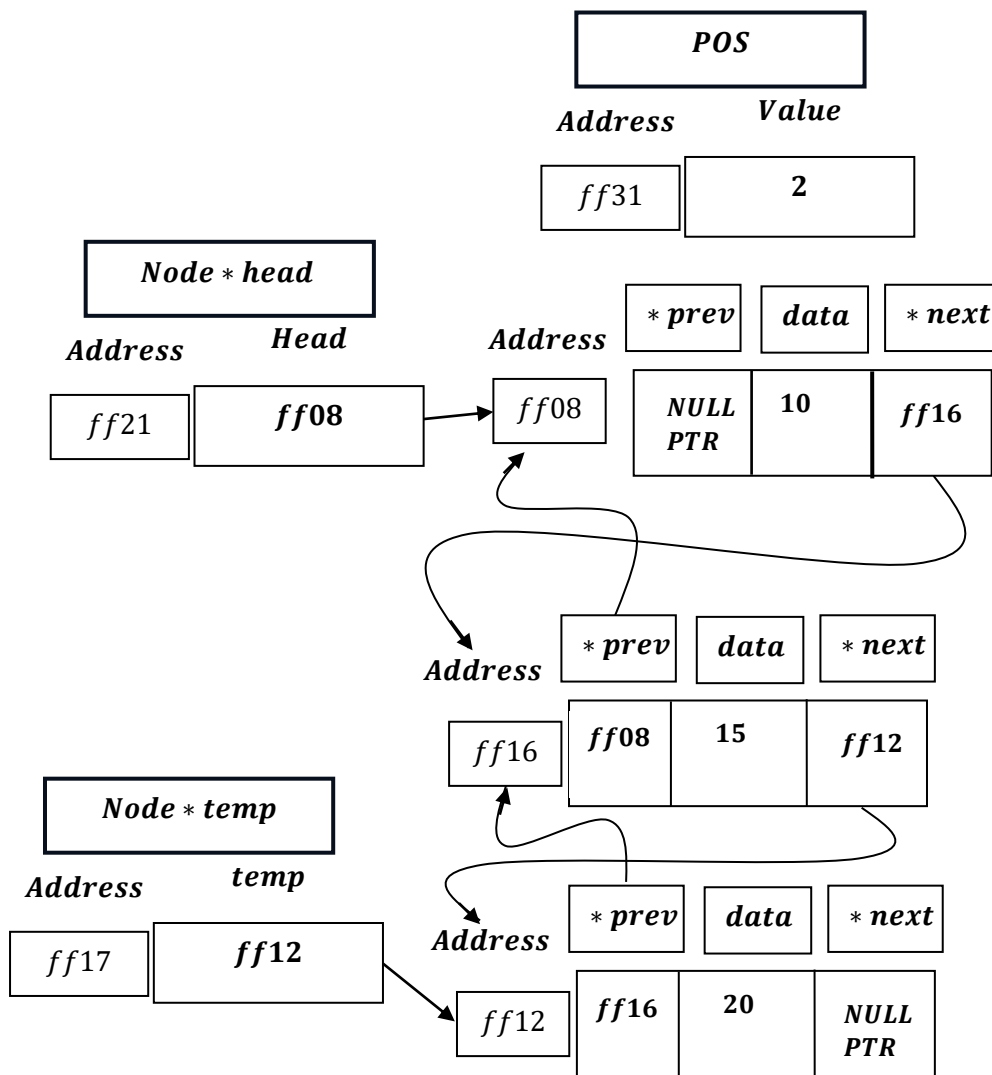


$$POS = POS + 1 = 1 + 1 = 2$$

Temp $\neq$ *NULLPTR* is true , hence:

$\rightarrow temp \rightarrow data = 15 \neq data = 25.$

$\rightarrow temp = temp \rightarrow next = ff12$

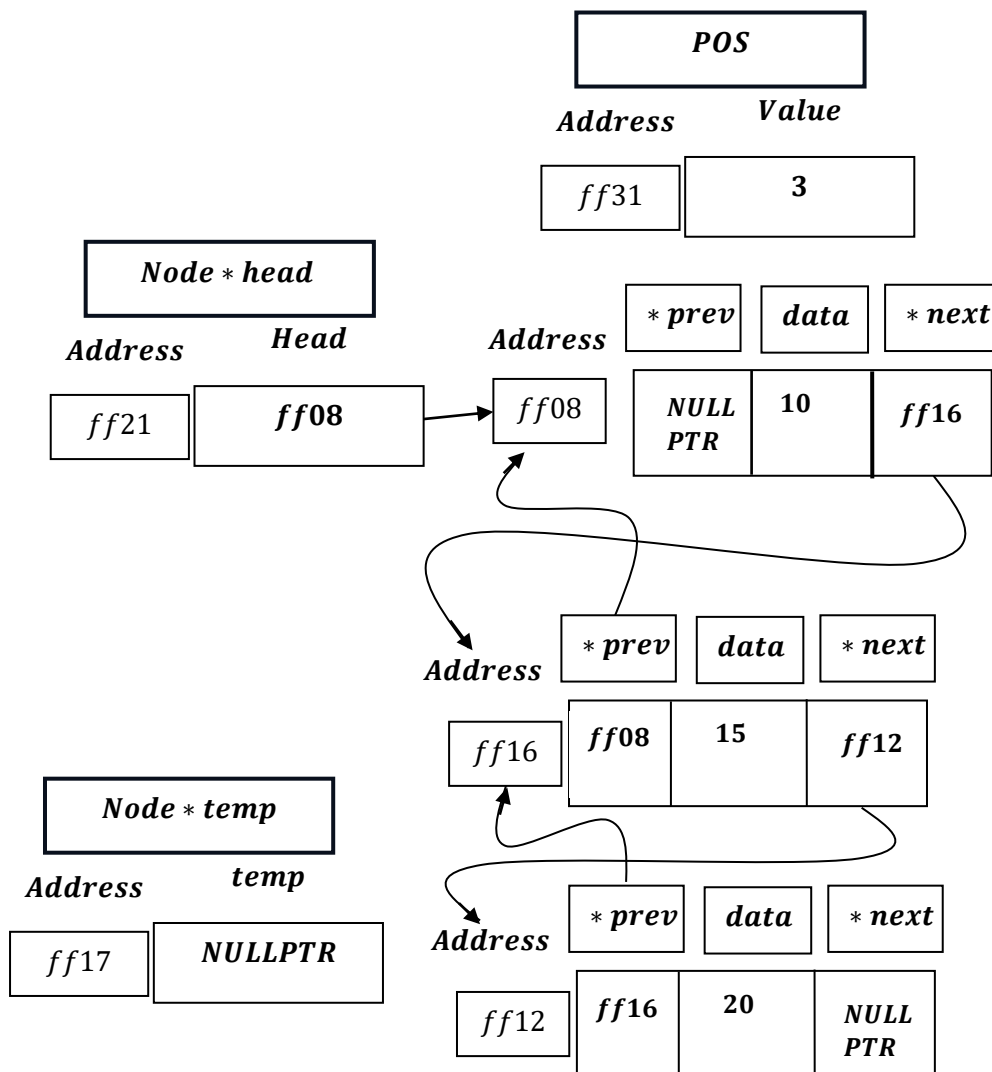


$$POS = POS + 1 = 2 + 1 = 3$$

Temp $\neq$ *NULLPTR* is true , hence:

$\rightarrow temp \rightarrow data = 20 \neq data = 25.$

$\rightarrow temp = temp \rightarrow next = NULLPTR$



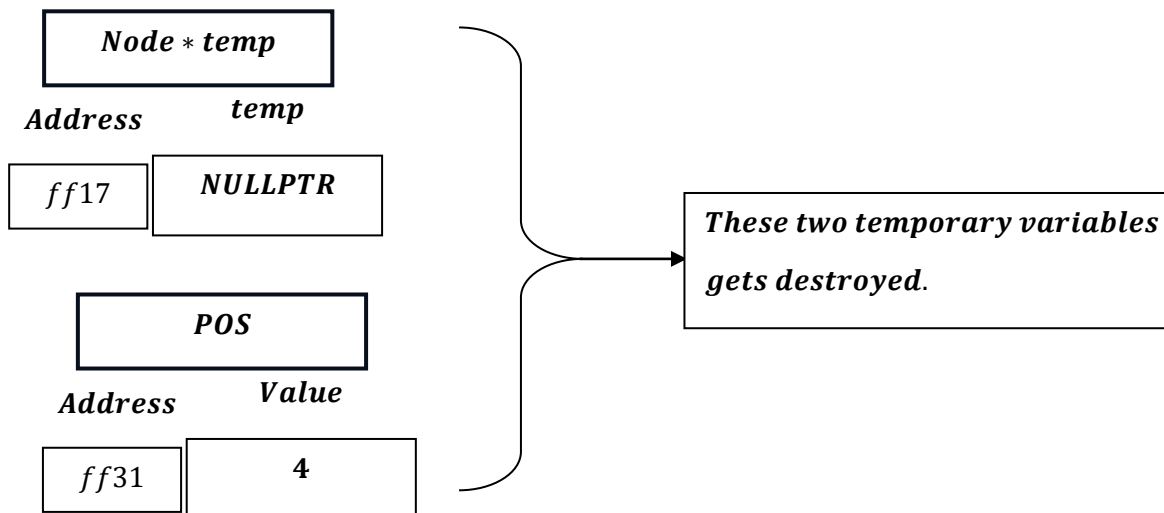
$POS = POS + 1 = 3 + 1 = 4.$

Now, $While(TEMP \neq NULLPTR)$ becomes false and it exits from loop. And, it prints:

```
cout << "Element " << data << " not found in the list."
<< endl;
```

i. e., Element 25 not found in the list.

As it exit from function Temp pointer along with pos variable gets destroyed:



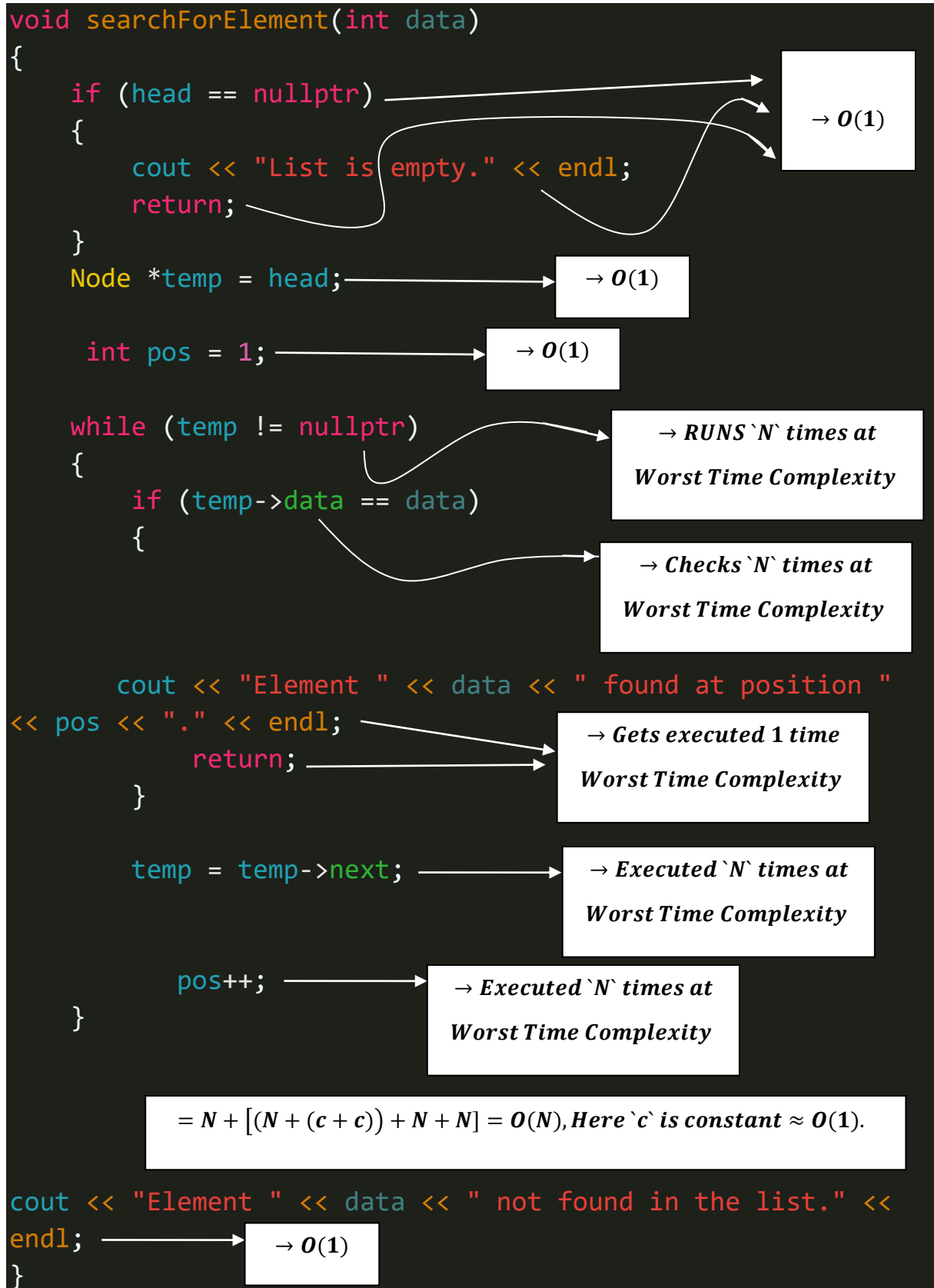
Doubly Linked List – Search For Element [Working Summary]:

This C++ function, `searchForElement`, is designed to find a specific integer within a singly linked list.

Working Summary

1. **Handles Empty List:** It first checks if the list is empty by seeing if the `head` pointer is `nullptr`. If it is, the function prints "List is empty." and exits.
 2. **Initializes Traversal:** If the list is not empty, it creates a temporary pointer `temp` that starts at the `head` of the list and initializes a position counter `pos` to 1.
 3. **Iterates Through the List:** The function enters a `while` loop that continues as long as `temp` is not `nullptr` (i.e., it hasn't reached the end of the list). In each iteration:
 - It **compares** the `data` in the current node (`temp->data`) with the `data` value being searched for.
 - **If a match is found**, it prints a success message with the element and its position, then immediately exits the function using `return`.
 - **If there's no match**, it moves to the next node (`temp = temp->next`) and increments the position counter (`pos++`).
 4. **Handles Element Not Found:** If the `while` loop finishes without finding the element (meaning `temp` becomes `nullptr`), the function prints a message indicating that the element was not found in the list.
-

Time Complexity of Search For Element Function Module



Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes, $O(1)$ time.

Worst Case (When Element Not Found)

→ If head = nullptr which is false, this checking takes constant time. Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer pointing to, this process takes constant amount of time i.e. $O(1)$.

→ integer pos variable assigned to 1. This process takes constant amount of time i.e. $O(1)$.

→ While loop condition : temp $\neq$ NULLPTR checks `N` times:

→ if condition checks data of the memory location pointed by temp pointer equals to user input data, this check takes `N` times. As condition is: ``Element not Found``, the condition never becomes true hence if block statement never runs.

→ Temp = Temp → next; runs N times.

→ POS ++ i.e. POS increment runs N times.

Hence total time complexity = $N + (N + N + N) = N + (3N) = O(N) + 3 O(N) = O(N)$.

→ Next, it prints `` Element [data] not found in the list. `` , this process takes constant amount of time i.e. $O(1)$.

Hence ,total time complexity : $O(1) + O(1) + O(1) + O(N) + O(1) = O(N)$.

Average Case

(If the elements in the linked list are uniformly distributed and the Element/key is equally likely to be any element in the list)

- if key is found at 1st Node , loop runs 1 time .***
- if key is found at 2nd Node , loop runs 2 time .***
- .***
- .***
- if key is found at Nth Node, loop runs N times.***

As , the key is equally likely to be any element in the list,

probablity of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

probablity of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

probablity of finding Nth element (p_2), if key equals to Nth element = $\frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the loops executes × probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned}\therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \\ &= \frac{N+1}{2} \\ &= \frac{N}{2} + \frac{1}{2}\end{aligned}$$

Applying , Big Theta Θ we get :

$$\begin{aligned}&= \frac{1}{2} \times (\Theta(N) + \Theta(1)) \\ &= \Theta(N) + \Theta(1) \\ &= \Theta(N).\end{aligned}$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Search Element For Element</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$
